Guion de demo — Banorte AI Finanzas (5 pasos)

Duración objetivo: **3–5 minutos** (medición del ensayo: **3:06** sobre el commit `287247d`). Todo corre sobre datos sintéticos y una base SQLite local; ninguna escritura mueve dinero real.

El catálogo A2UI tiene **9 componentes** por decisión de producto: los 5 originales más `PlanDePago`, `LineChart`, `Slider` y `BudgetAllocator`. Ninguno se retiró.

Preparación

1. Desde la raíz del repositorio, exporta `DEV_MODE` y arranca:

```
$env:DEV_MODE = "true"
scripts\start-all.cmd
```

`DEV_MODE=true` habilita los endpoints de degradación del paso 5. Sin esta variable, `POST /api/dev/provider/degrade` y `/recheck` devuelven **404**. `scripts\start-all.cmd` no la define por sí solo: hay que exportarla en la misma consola antes de arrancar.

El script propaga `BANORTE_DEMO_TODAY` al backend, al servidor MCP y al frontend, de modo que "hoy" es la misma fecha en toda la demo.

1. Abre la aplicación y entra a **Asistente** (segundo elemento del menú).
2. Comprueba que el indicador de salud diga **IA activa**. Si dice *Ayuda local*,

el paso 5 ya está ocurriendo: explica la degradación y continúa.

1. Ten a la mano `docs/DEMO-SCRIPT.md` y no escribas mensajes distintos a los

indicados: cada frase está probada contra el catálogo A2UI.

Paso 1 — Saldo (lectura, sin confirmación)

Mensaje exacto:

```
¿Cuánto dinero tengo?
```

Componente esperado: `StatCard` con el **Saldo total** en cuentas de demostración.

Narración:

> "El asistente no inventa la cifra: consulta el saldo con la herramienta MCP `get_balance`, que lo calcula en centavos enteros en la capa de dominio. La respuesta es solo lectura y no pide confirmación."

Qué mirar: la tarjeta muestra el mismo importe que la pantalla *Inicio*.

Paso 2 — Análisis (tendencia de gasto)

Mensaje exacto:

```
¿Cómo han cambiado mis gastos en los últimos 6 meses?
```

Componente esperado: `LineChart` con la serie mensual de gasto y su periodo declarado.

Narración:

> "Cada punto viene de `get_spending_trend`. El modelo elige el componente y copia la serie; no recalcula porcentajes ni sumas. El plan se valida contra los resultados de la herramienta antes de renderizarse, así que una cifra inventada se rechaza."

Qué mirar: el periodo visible y la nota con el cambio entre el primer y el último mes.

Paso 3 — Meta (avance de ahorro)

Mensaje exacto:

```
¿Cómo van mis metas de ahorro?
```

Componente esperado: `ProgressRing` con el avance de una meta sembrada (*Fondo de emergencia* o *Viaje a Japón*).

Narración:

> "El avance también lo calcula el dominio: ahorrado entre objetivo. El asistente solo lo presenta."

Qué mirar: el anillo, el monto ahorrado y el objetivo.

Paso 4 — Acción (ciclo cerrado con confirmación)

Mensaje exacto:

```
Quiero abonar $500 a Viaje a Japón.
```

Componente esperado: `TransferForm` con el monto, la meta y el texto de confirmación. El servidor ya emitió `confirmation_token` e `idempotency_key`.

Narración:

> "Preparar el abono no escribe nada. El formulario es la propuesta; el botón envía un evento de interfaz verificado en el servidor. Solo entonces el dominio aplica la transferencia, de forma atómica e idempotente."

Acción en pantalla: pulsa **Confirmar transferencia**.

Resultado esperado: el asistente responde *"Listo. Registré un abono de $500.00 a «Viaje a Japón»..."* y el nuevo `ui_plan` muestra:

- un `StatCard` **Abono aplicado** por `$500`, y
- un `ProgressRing` actualizado.

Narración de cierre:

> "El saldo de la meta subió exactamente $500 y se registró una transferencia con sus dos piernas. La reconciliación de invariantes sigue en verde. Si se repite el clic, la clave de idempotencia evita duplicar el movimiento."

Verificación opcional: abre *Ahorro* y confirma que *Viaje a Japón* subió $500; abre *Movimientos* y verifica una sola transferencia.

Paso 5 — Degradación controlada (sin reiniciar el backend)

Objetivo: demostrar que el producto sigue respondiendo con el motor local cuando el proveedor de IA no está disponible, sin depender de matar procesos ni de tener Ollama instalado.

Requisito: backend arrancado con `DEV_MODE=true`. Los endpoints de degradación solo existen en ese modo; con `DEV_MODE` apagado devuelven `404`.

```
$env:DEV_MODE = "true"
scripts\start-all.cmd
```

Antes del ensayo: consulta el estado real del proveedor:

```
Invoke-RestMethod http://127.0.0.1:8000/api/health |
  Select-Object -ExpandProperty provider | ConvertTo-Json
```

Caída simulada: sin tocar el backend ni Ollama, fuerza la degradación:

```
Invoke-RestMethod -Method Post `
  -Uri http://127.0.0.1:8000/api/dev/provider/degrade | ConvertTo-Json -Depth 4
```

Esperado: `forced = true`, `provider.mode = "demo"` y `provider.reason = "provider_unavailable"`.

Mensaje exacto: pide una respuesta en modo `think`:

```
$body = @{
  message = "¿Cuál es mi saldo?"
  mode = "think"
  session_id = "demo-degraded"
} | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/api/agent/chat `
  -ContentType "application/json" -Body $body |
  Select-Object mode, degraded, reason | ConvertTo-Json
```

Resultado esperado: la petición no devuelve 500. Devuelve:

- `mode = "demo"`,
- `degraded = true`,
- `reason = "provider_unavailable"`,
- un `ui_plan` válido con `StatCard` de saldo.

El camino es el mismo de una caída real: `loop.get_provider()` devuelve `None` y `_demo_fallback(...)` responde con las herramientas MCP (`backend/app/services/agent/loop.py:298-302`). La cifra sigue viniendo de la misma herramienta MCP.

Recuperación: retira la degradación forzada y vuelve a comprobar el proveedor real:

```
Invoke-RestMethod -Method Post `
  -Uri http://127.0.0.1:8000/api/dev/provider/recheck | ConvertTo-Json -Depth 4
```

Esperado: `forced = false` y `provider.mode = "live"` si hay proveedor configurado. Sin proveedor, el healthcheck reporta `mode = "demo"` con `reason = "not_configured"`, que es el estado real y no una caída.

Implementación: `backend/app/services/agent/providers.py` expone `force_degraded()` y `clear_forced_degraded()`; los endpoints viven en `backend/app/api/routes.py` y están protegidos por `config.DEV_MODE`.

Alcance de la interfaz: el cliente web actual envía `mode=demo` (`frontend/src/components/AgentView.tsx:82-87`), de modo que este paso se demuestra contra la API con `mode=think`; la interfaz sigue usando el motor local.

Cierre (30 segundos)

> "Todo lo que vieron es verificable: catálogo A2UI estricto, cifras fundamentadas en herramientas MCP, escrituras solo tras confirmación verificada y una suite que compara el hash de la base real antes y después para probar que las pruebas no la tocan."

Mensajes de respaldo (si el presentador se adelanta)

Paso	Mensaje	Componente
1	`¿Cuál es mi saldo?`	`StatCard`
2	`¿En qué gasté este mes?`	`BarChart`
3	`Muéstrame mis últimos movimientos.`	`Table`
4	`Quiero abonar $500 a Fondo de emergencia.`	`TransferForm`

No cambies el monto ni el nombre de la meta en el paso 4 sin volver a ensayar: la confirmación valida monto, meta, token e idempotencia contra el estado del servidor.

Tiempos medidos (commit 287247d)

Medición sobre el commit `287247d`, con ensayo narrado usando voz sintetizada es-MX y pausas humanas de interacción. **No hubo presentador físico**; los tiempos incluyen narración real medida y maniobras de tipeo, clic y cambio de ventana.

Bloque	Tiempo
Preparación	12.0 s
Paso 1 — Saldo	23.4 s
Paso 2 — Tendencia	28.4 s
Paso 3 — Metas	15.2 s
Paso 4 — Acción	51.2 s
Paso 5 — Degradación	34.2 s
Cierre	21.7 s
Suma pasos 1–5	**152.4 s (2:32)**
Total con preparación y cierre	**186.4 s (3:06)**

Ningún paso supera 60 s. El más largo es el paso 4 con 51.2 s.

Ajuste opcional si hay retraso: omitir la verificación de *Ahorro* y *Movimientos* del paso 4 (~5 s) y dejar la pestaña *Movimientos* abierta antes de empezar. El `StatCard` y el `ProgressRing` de la propia respuesta ya prueban el abono.

Plan B si falla el proyector

Actívalo si no hay señal, el proyector no reconoce la laptop, los colores no se leen o la máquina de demo falla. La demo se puede sostener con capturas y narración, sin interacción en vivo.

Material de respaldo

- `docs/demo-fallback/README.md` — cómo generar y nombrar las capturas.
- `docs/demo-fallback/guion-demo.pdf` — guion completo para leer en una segunda

pantalla o imprimir.

- `docs/demo-fallback/paso1-saldo.png`
- `docs/demo-fallback/paso2-tendencia.png`
- `docs/demo-fallback/paso3-metas.png`
- `docs/demo-fallback/paso4-formulario.png`
- `docs/demo-fallback/paso4-resultado.png`
- `docs/demo-fallback/paso5-degradacion.png`

a) Capturas de los 5 pasos

Genera las capturas antes de la demo siguiendo `docs/demo-fallback/README.md`. Resumen:

1. Arranca la demo normal con `DEV_MODE=true`.
2. Ejecuta cada paso con el mensaje exacto de este guion.
3. Con la respuesta en pantalla, pulsa `Win + Shift + S`, recorta la tarjeta o

el panel completo y guarda con el nombre indicado en `docs/demo-fallback/`.

1. En el paso 4 guarda **dos** capturas: el `TransferForm` antes de confirmar y

el `StatCard` + `ProgressRing` después de confirmar.

1. En el paso 5 fuerza la degradación, recarga para que el indicador de salud se

actualice y captura la respuesta con el aviso **Ayuda local (IA no disponible)**. El motivo `provider_unavailable` viaja en el JSON de la API, no en la interfaz; si hace falta, captura también la terminal.

No incluyas claves, rutas personales ni datos fuera de la base sintética.

b) PDF para segunda pantalla

`docs/demo-fallback/guion-demo.pdf` se genera a partir de este mismo documento. Si cambia el guion, regenéralo antes de la demo:

```
# Opción 1: abrir este Markdown en el navegador y usar Imprimir → Guardar como PDF.
# Opción 2: con Edge instalado, imprimir el HTML renderizado a PDF sin abrir ventanas.
```

También sirve abrir este `DEMO-SCRIPT.md` en una segunda pantalla; el PDF es solo para que la lectura sea estable y sin dependencia del editor.

c) Guion si solo hay capturas

Paso	Qué decir mientras se muestra la captura
1	"El asistente consulta el saldo con `get_balance`; la cifra se calcula en centavos en el dominio y la respuesta es de solo lectura."
2	"Cada punto viene de `get_spending_trend`. El modelo elige el componente y copia la serie; no recalcula porcentajes ni sumas. El plan se valida contra la herramienta antes de renderizarse."
3	"El avance lo calcula el dominio: ahorrado entre objetivo. El asistente solo lo presenta."
4	"Preparar el abono no escribe nada. Esta es la propuesta; al confirmar, el servidor verifica token e idempotencia y solo entonces el dominio aplica la transferencia. El resultado muestra el `StatCard` y el `ProgressRing` de Viaje a Japón."
5	"Miren el indicador superior derecho: dice «Ayuda local (IA no disponible)». Las cifras son las mismas que en el paso 1 porque el motor determinista siguió respondiendo. Forzamos la caída del proveedor. La petición no devuelve error: responde con el motor local, `degraded=true` y motivo `provider_unavailable`, con las mismas cifras de la herramienta MCP. Después se retira la degradación."

Cómo cambiar a "modo capturas" sin perder el hilo

1. No reinicies la narrativa: di "les muestro la misma pantalla en captura" y cambia a la carpeta o al PDF.
1. Sigue el orden 1 → 5. Cada captura sustituye la interacción, no la explicación.
1. En el paso 4 muestra primero el formulario y luego el resultado; explica que entre ambas imágenes ocurrió la confirmación verificada.
1. En el paso 5 muestra la respuesta degradada y aclara que el `recheck` ya se ejecutó, aunque no se vea en vivo.
1. Si se recupera el proyector, vuelve a la demo en vivo en el paso siguiente; no repitas los pasos ya narrados.